```
        RuleFor(x => x.Quantity)
            .NotNull()
            .InclusiveBetween(1, 1000);                                    ❼
    }
}
```

❶ The validator inherits from AbstractValidator.

❷ Defines the static list of currency codes that are supported

❸ You define validation rules in the validator's constructor.

❹ RuleFor is used to add a new validation rule. The lambda syntax allows for strong typing.

❺ There are equivalent rules for common DataAnnotations validation attributes.

❻ You can easily add custom validation rules without having to create separate classes.

❼ Thanks to strong typing, the rules available depend on the property being validated.

Your first impression of this code might be that it's quite verbose compared with listing 32.7, but remember that listing 32.7 used a custom validation attribute, `[CurrencyCode]`. The validation in listing 32.11 doesn't require anything else. The logic implemented by the `[CurrencyCode]` attribute is right there in the validator, making it easy to reason about. The `Must()` method can be used to perform arbitrarily complex validations without having the additional layers of indirection required by custom `DataAnnotations` attributes.

On top of that, you'll notice that you can define only validation rules that make sense for the property being validated. Previously, there was nothing to stop us from applying the `[CurrencyCode]` attribute to the `Quantity` property; that's not possible with FluentValidation.

Of course, just because you can write the custom `[CurrencyCode]` logic in-line doesn't necessarily mean you have to. If a rule is used in multiple parts of your application, it may make sense to extract it into a helper class. The following listing shows how you could extract the currency code logic into an extension method that can be used in multiple validators.